

Import Data Using Transform Maps (Spreadsheet)

Full Stack Development with MERN - Project Documentation

Team ID: 6ab7f10630f9767f87728ca3

Team Leader: E Dilli Kumar

1. Introduction

Project Title: Import Data Using Transform Maps (Spreadsheet)

Team ID: 6ab7f10630f9767f87728ca3

Team Members

Name	Role
E Dilli Kumar	Team Leader - project planning, backend architecture, transform engine, code review
R Vinish	Team Member - Node.js / Express API, file upload and spreadsheet parsing
K Jerina	Team Member - MongoDB schemas, transform map logic and validation
B Mohanavalli	Team Member - testing, data verification and bug tracking
P Sri Ragavan	Team Member - React frontend, UI design and documentation

2. Project Overview

Purpose

Data is often shared as Excel/CSV spreadsheets, and typing it into an application row by row is slow and error-prone. This MERN web application lets a user upload a spreadsheet, define a **Transform Map** (which spreadsheet column goes to which database field, with type conversion and validation), preview the result, and import the data into MongoDB in one step.

Goals: automate bulk imports, avoid duplicates, validate data during the load, and keep a history of every import.

Features

- Upload .xlsx / .xls / .csv files from the React interface.
- Automatic reading of headers and a preview of the first rows.
- Transform Map editor: map source columns to target fields, set data type, required flag and default value.
- Save, edit and reuse Transform Maps for future files with the same layout.
- Coalesce (unique key) support: existing records are updated, new ones inserted - no duplicates.
- Row-level validation with an error report for rejected rows.
- Import history with counts of total, inserted, updated and failed rows.
- JWT-based login with admin and user roles.

3. Architecture

Frontend (React)

- Single-page application built with React and React Router.
- **Pages:** Login, Dashboard, Upload & Map, Transform Maps, Import History, Records.
- **Components:** FileUploader, PreviewTable, MappingEditor, ImportSummary, Navbar, ProtectedRoute.

- **State:** React hooks (useState, useEffect) and Context API for the logged-in user and token.
- **API calls:** Axios instance in services/api.js that attaches the JWT in the Authorization header.

Backend (Node.js and Express.js)

- REST API built with Express.js; routes -> controllers -> services pattern.
- **multer** receives the uploaded file (memory storage, size and type limits).
- **xlsx (SheetJS)** parses the workbook into JSON rows.
- **Transform service** applies the saved Transform Map: renames columns, converts types, applies defaults, validates required fields.
- **Bulk upsert** into MongoDB with Mongoose bulkWrite using the coalesce field.
- Middleware: JWT auth, role check, error handler, CORS.

Database (MongoDB)

MongoDB stores users, transform maps, import jobs and the imported records. Mongoose models are used for schema and validation.

Collection	Fields	Purpose
users	name, email, password (hashed), role, createdAt	Login and authorization
transformmaps	name, targetCollection, coalesceField, fieldMaps[{source, target, type, required, defaultValue}], createdBy	Stores column-to-field mapping rules
importjobs	fileName, transformMap (ref), user (ref), total, inserted, updated, failed, errors[{row, message}], createdAt	History of every import
employees (sample target)	empId (unique), name, email, department, location, joiningDate	Data loaded from spreadsheet

Data flow: React upload → Express + multer → SheetJS parse → Transform Map applied → Mongoose bulkWrite (upsert) → MongoDB → summary returned to React.

4. Setup Instructions

Prerequisites

- Node.js (v18 or later) and npm
- MongoDB (local installation) or a MongoDB Atlas cluster
- Git
- Code editor (VS Code) and a browser
- Postman (optional, for API testing)

Installation

1. Clone the repository: `git clone <your-repo-url>` and open the project folder.
2. Install backend packages: `cd server && npm install`
3. Install frontend packages: `cd ../client && npm install`
4. Create server/.env with the variables shown below.
5. Create client/.env with `REACT_APP_API_URL=http://localhost:5000/api`.
6. Make sure MongoDB is running (or the Atlas connection string is correct).

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/transformmaps
JWT_SECRET=your_secret_key
JWT_EXPIRES_IN=1d
MAX_FILE_SIZE_MB=5
```

5. Folder Structure

Client (React)

```
client/
|-- public/
'-- src/
    |-- components/    FileUploader, PreviewTable, MappingEditor,
    |                  ImportSummary, Navbar, ProtectedRoute
    |-- pages/         Login, Dashboard, UploadImport, Maps, History, Records
    |-- context/       AuthContext.js
    |-- services/      api.js (Axios instance)
    |-- App.js
    '-- index.js
```

Server (Node.js / Express)

```
server/
|-- config/           db.js
|-- models/           User.js, TransformMap.js, ImportJob.js, Employee.js
|-- routes/           auth.js, maps.js, imports.js, records.js
|-- controllers/      authController, mapController, importController
|-- middleware/       auth.js, role.js, upload.js, errorHandler.js
|-- services/         parseService.js, transformService.js
|-- tests/            auth.test.js, transform.test.js, import.test.js
|-- .env
'-- server.js
```

6. Running the Application

Open two terminals:

Server	Command
Backend (from server/)	npm start - runs on http://localhost:5000
Frontend (from client/)	npm start - runs on http://localhost:3000

Open <http://localhost:3000>, register or log in, upload a spreadsheet, create or choose a Transform Map and click Import.

7. API Documentation

Base URL: <http://localhost:5000/api>. All routes except auth require the header Authorization: Bearer <token>.

Method	Endpoint	Description / parameters
POST	/auth/register	Body: name, email, password. Creates a user.
POST	/auth/login	Body: email, password. Returns JWT token and user.
GET	/maps	List all saved Transform Maps.

Method	Endpoint	Description / parameters
POST	/maps	Body: name, targetCollection, coalesceField, fieldMaps[]. Create a map.
PUT	/maps/:id	Update a Transform Map.
DELETE	/maps/:id	Delete a Transform Map (admin).
POST	/imports/preview	multipart/form-data: file. Returns headers and first 10 rows.
POST	/imports/upload	multipart/form-data: file, mapId. Runs the import and returns summary.
GET	/imports	Import history of the logged-in user.
GET	/imports/:id	Details and row errors of one import.
GET	/records	List imported records (query: page, limit, search).

Example: POST /auth/login

```
Request: { "email": "admin@example.com", "password": "Pass@123" }
Response: { "token": "eyJhbGciOi...", "user": { "id": "66f1...", "name": "Admin", "role": "admin" }
```

Example: POST /imports/upload

```
Response (200):
{
  "importId": "66f2a1c9...",
  "fileName": "employee_data.xlsx",
  "total": 50,
  "inserted": 42,
  "updated": 6,
  "failed": 2,
  "errors": [ { "row": 7, "message": "name is required" } ]
}
```

Example: POST /maps

```
{ "name": "Employee Map", "targetCollection": "employees", "coalesceField": "empId",
  "fieldMaps": [
    { "source": "Emp ID", "target": "empId", "type": "String", "required": true },
    { "source": "Name", "target": "name", "type": "String", "required": true },
    { "source": "Joining Date", "target": "joiningDate", "type": "Date" } ] }
```

8. Authentication

- **Registration/Login:** passwords are hashed with bcrypt before saving in MongoDB.
- **Tokens:** on login the server signs a JSON Web Token (JWT) containing the user id and role, valid for the time set in JWT_EXPIRES_IN.
- **Client:** the token is stored in the browser and sent as a Bearer token in every request by the Axios instance.
- **Protected routes:** an auth middleware verifies the token; invalid or expired tokens receive 401.
- **Authorization:** role middleware - normal users can upload, import and view their history; admins can also delete maps and view all imports.
- React ProtectedRoute redirects unauthenticated users to the Login page.

9. User Interface

Insert screenshots of these screens under their captions:

- Login / Register page
- Dashboard with import statistics
- Upload page with file preview table
- Transform Map editor (column-to-field mapping)
- Import summary with inserted / updated / failed counts
- Import history and Records list

10. Testing

A combination of automated and manual testing was used.

Level	Tool	What is tested
Unit	Jest	transformService: renaming, type conversion, defaults, required-field validation
API	Jest + Supertest, Postman	Auth, maps CRUD, preview and upload endpoints, 401/403 responses
Frontend	React Testing Library	Rendering of upload form, preview table and mapping editor
Manual	Browser	Full flow with valid, duplicate, blank and wrong-heading spreadsheets

11. Screenshots or Demo

Add screenshots of the working application here.

Demo link: [Insert link]

GitHub link: [Insert link]

12. Known Issues

- Spreadsheet headings must match the names used in the Transform Map.
- Very large files (over the size limit) are rejected; they must be split.
- Excel date cells with mixed formats may need manual correction.
- Only the first sheet of a workbook is imported.

13. Future Enhancements

- Background processing of large files with a job queue and progress bar.
- Support for multiple target collections and reference lookups.
- Auto-suggest column mapping using header similarity.
- Downloadable error report (Excel) for failed rows.
- Scheduled imports from cloud storage or SFTP.
- Undo/rollback of an import.